

Zeros de Funções, Derivadas Numéricas e o Método de Newton

Fundamentos e Aplicações em Ciência da Computação

EXA1132 – Cálculo Numérico

Contents

1	Introdução	2
2	Zeros de Funções	2
2.1	Existência de raízes: Teorema de Bolzano	2
3	Derivadas Numéricas: Diferenças Finitas	3
3.1	Dedução via Série de Taylor	3
3.1.1	Diferença progressiva (<i>forward</i>)	3
3.1.2	Diferença regressiva (<i>backward</i>)	3
3.1.3	Diferença centrada (<i>central</i>)	3
3.2	Derivada segunda por diferenças finitas	3
3.3	O dilema da escolha de h	4
3.4	Implementação em Python	4
4	Método de Newton-Raphson	4
4.1	Dedução geométrica	4
4.2	Algoritmo	5
4.3	Convergência quadrática	5
4.4	Método de Newton com derivada numérica	5
4.5	Implementação em Python	6
5	Aplicações em Ciência da Computação	6
5.1	Otimização e Aprendizado de Máquina	6
5.2	Computação Gráfica e Visão Computacional	7
5.3	Compiladores e Sistemas	7
5.4	Criptografia e Teoria dos Números	7
5.5	Simulação e Jogos	7
6	Exercícios Propostos	8
	Referências	8

1 Introdução

Grande parte dos problemas de engenharia, física e computação recaem, em algum momento, sobre uma pergunta simples de enunciar mas nem sempre fácil de responder: *para quais valores de x a função $f(x)$ se anula?* Encontrar essas raízes – os **zeros da função** – é uma tarefa central do Cálculo Numérico, pois problemas de otimização, ajuste de curvas, resolução de equações não lineares, calibração de modelos e treinamento de algoritmos de aprendizado de máquina são, em última análise, problemas de busca de zeros (ou de pontos críticos, que são zeros da derivada).

Este material apresenta três blocos interligados:

- (i) a formalização do problema de zeros de funções;
- (ii) o cálculo aproximado de derivadas por **diferenças finitas**, necessário quando não se dispõe da expressão analítica de $f'(x)$;
- (iii) o **método de Newton-Raphson**, que combina os dois blocos anteriores para construir um algoritmo de convergência rápida.

Ao final, discutimos aplicações concretas em Ciência da Computação, mostrando que esses métodos não são um exercício isolado de disciplina, mas ferramentas que aparecem em compiladores, motores gráficos, algoritmos de aprendizado de máquina e sistemas de criptografia.

2 Zeros de Funções

Definição 2.1 (Zero ou raiz de uma função) *Seja $f : [a, b] \rightarrow \mathbb{R}$ uma função contínua. Dizemos que $\xi \in [a, b]$ é um **zero** (ou **raiz**) de f se*

$$f(\xi) = 0.$$

Na prática, não obtemos ξ de forma exata: construímos uma sequência $x_0, x_1, x_2, \dots$ que converge para ξ , parando quando um critério de tolerância ε é satisfeito.

2.1 Existência de raízes: Teorema de Bolzano

Teorema 2.2 (Teorema do Valor Intermediário / Bolzano) *Se f é contínua em $[a, b]$ e $f(a) \cdot f(b) < 0$, então existe pelo menos um $\xi \in (a, b)$ tal que $f(\xi) = 0$.*

Esse resultado garante *existência*, não *unicidade*: pode haver mais de uma raiz no intervalo. Por isso, o isolamento da raiz (via tabelamento de f ou análise gráfica) é sempre o primeiro passo antes de aplicar um método iterativo.

Observação 2.3 *Métodos como bisseção e falsa posição exploram diretamente o Teorema de Bolzano (métodos intervalares, sempre convergentes). O método de Newton, que veremos na Seção 4, é um método pontual: mais rápido, porém sem garantia incondicional de convergência.*

3 Derivadas Numéricas: Diferenças Finitas

Em muitas situações práticas – funções definidas apenas por tabelas de dados, saídas de simulações, ou expressões complicadas demais para derivar analiticamente – precisamos estimar $f'(x)$ sem conhecer a fórmula exata da derivada. A ferramenta central para isso é a **expansão em série de Taylor**.

3.1 Dedução via Série de Taylor

Considere f suficientemente diferenciável em uma vizinhança de x , e um passo pequeno $h > 0$. As expansões de Taylor são:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + O(h^4) \quad (1)$$

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f'''(x) + O(h^4) \quad (2)$$

3.1.1 Diferença progressiva (*forward*)

Isolando $f'(x)$ em (1):

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}, \quad \text{erro } O(h). \quad (3)$$

3.1.2 Diferença regressiva (*backward*)

Analogamente, de (2):

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}, \quad \text{erro } O(h). \quad (4)$$

3.1.3 Diferença centrada (*central*)

Subtraindo (2) de (1), os termos em h^2 se cancelam:

$$\boxed{f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}}, \quad \text{erro } O(h^2). \quad (5)$$

A fórmula centrada é preferida sempre que f estiver definida dos dois lados de x : o erro cai quadraticamente com h , em vez de linearmente.

3.2 Derivada segunda por diferenças finitas

Somando (1) e (2):

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}, \quad \text{erro } O(h^2). \quad (6)$$

Essa fórmula é a base de métodos de diferenças finitas para EDOs e EDPs (por exemplo, discretização da equação do calor ou da equação de Poisson).

3.3 O dilema da escolha de h

O erro total de uma derivada numérica tem duas fontes que competem entre si:

- **Erro de truncamento:** decorrente de cortar a série de Taylor; *diminui* quando $h \rightarrow 0$.
- **Erro de arredondamento:** decorrente da aritmética de ponto flutuante ao subtrair dois números muito próximos ($f(x+h) - f(x-h)$), fenômeno de **cancelamento catastrófico**; *aumenta* quando $h \rightarrow 0$.

Observação 3.1 *Existe um h ótimo que minimiza o erro total. Para a diferença centrada em aritmética de precisão dupla ($\epsilon_{máquina} \approx 10^{-16}$), uma boa estimativa prática é $h_{ótimo} \approx \sqrt[3]{\epsilon_{máquina}} \approx 10^{-5}$ a 10^{-6} . Escolher h absurdamente pequeno (ex. 10^{-15}) tipicamente piora o resultado.*

3.4 Implementação em Python

Listing 1: Diferenças finitas em Python

```
1 def derivada_progressiva(f, x, h=1e-5):
2     return (f(x + h) - f(x)) / h
3
4 def derivada_regressiva(f, x, h=1e-5):
5     return (f(x) - f(x - h)) / h
6
7 def derivada_centrada(f, x, h=1e-5):
8     return (f(x + h) - f(x - h)) / (2 * h)
9
10 def segunda_derivada(f, x, h=1e-4):
11     return (f(x + h) - 2 * f(x) + f(x - h)) / h**2
```

4 Método de Newton-Raphson

4.1 Dedução geométrica

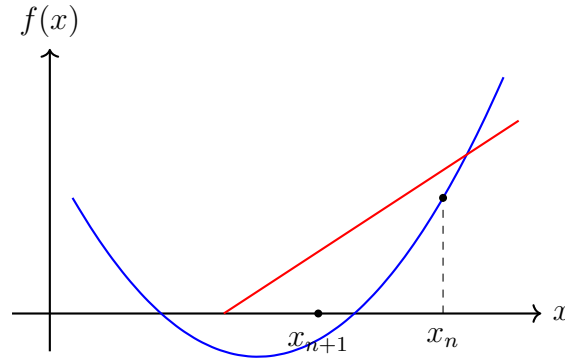
A ideia do método de Newton é simples e poderosa: partindo de uma aproximação x_n , traçamos a **reta tangente** ao gráfico de f nesse ponto e tomamos como próxima aproximação x_{n+1} o ponto onde essa reta cruza o eixo x .

A equação da reta tangente em x_n é

$$y = f(x_n) + f'(x_n)(x - x_n).$$

Impondo $y = 0$ e isolando x , obtemos a **fórmula de recorrência de Newton-Raphson**:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (7)$$



4.2 Algoritmo

1. Escolha uma aproximação inicial x_0 .
2. Para $n = 0, 1, 2, \dots$, calcule

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

3. Pare quando $|x_{n+1} - x_n| < \varepsilon$ **ou** $|f(x_{n+1})| < \varepsilon$ **ou** o número máximo de iterações for atingido.

4.3 Convergência quadrática

Teorema 4.1 (Convergência local do método de Newton) *Se $f \in C^2$ numa vizinhança de ξ , $f(\xi) = 0$, $f'(\xi) \neq 0$, e x_0 está suficientemente próximo de ξ , então a sequência (x_n) converge para ξ com **ordem quadrática**: existe uma constante $C > 0$ tal que*

$$|x_{n+1} - \xi| \leq C |x_n - \xi|^2.$$

Na prática, isso significa que o número de dígitos corretos *aproximadamente dobra* a cada iteração – daí a fama do método de Newton como um dos algoritmos de busca de raízes mais rápidos quando converge.

Observação 4.2 (Quando o método falha) *A convergência não é garantida globalmente. Problemas típicos:*

- $f'(x_n) \approx 0$ (*tangente quase horizontal*): o passo explode;
- ponto inicial x_0 longe da raiz: pode divergir ou oscilar;
- raízes múltiplas ($f'(\xi) = 0$): a convergência deixa de ser quadrática e passa a ser linear.

4.4 Método de Newton com derivada numérica

Quando $f'(x)$ não está disponível analiticamente, substituímos a derivada exata pela diferença centrada da Seção anterior:

$$x_{n+1} = x_n - \frac{f(x_n)}{\frac{f(x_n + h) - f(x_n - h)}{2h}}. \quad (8)$$

Isso conecta diretamente os dois primeiros blocos deste material e é a forma como o método de Newton é implementado quando se trabalha com funções “caixa-preta” (por exemplo, saída de uma simulação numérica ou de uma rede neural).

Observação 4.3 (Relação com o Método da Secante) *Se, em vez de usar diferenças finitas centradas, aproximarmos $f'(x_n)$ pela inclinação da reta que liga os dois últimos iterados,*

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}},$$

*obtemos o **Método da Secante** – uma variante que evita calcular f' a qualquer custo, ao preço de convergência superlinear (ordem $\approx 1,618$, a razão áurea) em vez de quadrática.*

4.5 Implementação em Python

Listing 2: Método de Newton (com derivada analítica e numérica)

```

1 def newton(f, df, x0, tol=1e-8, max_iter=100):
2     x = x0
3     for i in range(max_iter):
4         fx = f(x)
5         if abs(fx) < tol:
6             return x, i
7         dfx = df(x)
8         if abs(dfx) < 1e-14:
9             raise ZeroDivisionError("Derivada proxima de zero.")
10        x = x - fx / dfx
11        raise RuntimeError("Numero maximo de iteracoes excedido.")
12
13 def newton_derivada_numerica(f, x0, h=1e-5, tol=1e-8, max_iter=100)
14 :
15     df_num = lambda x: (f(x + h) - f(x - h)) / (2 * h)
16     return newton(f, df_num, x0, tol, max_iter)
17
18 # Exemplo: raiz de f(x) = x**3 - 2x - 5
19 f = lambda x: x**3 - 2*x - 5
20 df = lambda x: 3*x**2 - 2
21 raiz, iteracoes = newton(f, df, x0=2.0)
22 print(f"Raiz: {raiz:.10f} (em {iteracoes} iteracoes)")

```

5 Aplicações em Ciência da Computação

Zeros de funções, derivadas numéricas e o método de Newton estão muito mais presentes no dia a dia da Computação do que sugere sua aparência de tópico puramente matemático.

5.1 Otimização e Aprendizado de Máquina

Treinar um modelo de aprendizado de máquina é, em essência, encontrar os parâmetros θ que minimizam uma função de perda $L(\theta)$ – ou seja, encontrar os zeros do **gradiente** $\nabla L(\theta) = 0$.

- O algoritmo de **gradiente descendente**, base do treinamento de redes neurais, usa a derivada (via *backpropagation*, que é essencialmente a regra da cadeia aplicada de forma automática) para caminhar em direção a um mínimo.
- Métodos de **segunda ordem** (Newton, quase-Newton como BFGS e L-BFGS) usam a matriz Hessiana – aproximada por diferenças finitas quando não há forma fechada – para convergir muito mais rápido que o gradiente descendente simples perto do mínimo.
- Bibliotecas de *differentiable programming* (autograd, PyTorch, JAX) usam diferenciação automática, mas o teste e a validação desses gradientes frequentemente são feitos por **verificação numérica via diferenças finitas** (“*gradient checking*”).

5.2 Computação Gráfica e Visão Computacional

- **Ray marching / ray tracing**: interseções entre raios de luz e superfícies implícitas $f(x, y, z) = 0$ são encontradas por métodos tipo Newton.
- **Deteção de bordas** em processamento de imagens (filtros de Sobel, Prewitt) são, literalmente, aproximações discretas da derivada de uma imagem via diferenças finitas em uma grade de pixels.
- **Fast Inverse Square Root**: o famoso algoritmo do Quake III usa uma iteração de Newton para refinar rapidamente uma estimativa de $1/\sqrt{x}$, base de cálculos de iluminação e normalização de vetores em motores de jogos.

5.3 Compiladores e Sistemas

- Análise de **pontos fixos** em compiladores (análise de fluxo de dados, inferência de tipos) é conceitualmente um problema de busca de zero/ponto fixo de uma função sobre um reticulado.
- **Escalonamento e controle de sistemas** (ex.: controladores PID em sistemas embarcados e robótica) usam derivadas numéricas do sinal de erro para calcular o termo derivativo em tempo real.

5.4 Criptografia e Teoria dos Números

- O método de Newton adaptado a aritmética modular é usado no cálculo de **raízes quadradas modulares** e em rotinas de aritmética de precisão arbitrária (GMP, bibliotecas usadas em RSA), incluindo o cálculo eficiente de $1/x \pmod n$ via iteração de Newton-Raphson.

5.5 Simulação e Jogos

- Motores de física (deteção de colisão, *constraint solvers*) resolvem sistemas de equações não lineares a cada quadro, frequentemente com variantes do método de Newton adaptadas para múltiplas variáveis (Newton multivariado, $\mathbf{J}(\mathbf{x})^{-1}$ sendo a matriz Jacobiana).

- Simulações numéricas de EDPs (clima, fluidos, calor) discretizam derivadas espaciais e temporais via diferenças finitas, exatamente como na Seção 3.

6 Exercícios Propostos

1. Deduza a fórmula de diferença progressiva de segunda ordem,

$$f'(x) \approx \frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h},$$

mostrando que o erro de truncamento é $O(h^2)$.

2. Implemente em Python as três fórmulas de diferenças finitas (progressiva, regressiva, centrada) para $f(x) = \sin(x)$ em $x = \pi/4$, variando $h \in \{10^{-1}, 10^{-3}, 10^{-5}, 10^{-8}, 10^{-12}\}$. Compare com o valor exato $f'(\pi/4) = \cos(\pi/4)$ e identifique experimentalmente o fenômeno descrito na Seção 3.3.
3. Aplique o método de Newton para encontrar a raiz positiva de $f(x) = x^3 - 2x - 5$, partindo de $x_0 = 2$. Compare o número de iterações necessárias usando (a) a derivada analítica e (b) a derivada numérica centrada com $h = 10^{-5}$.
4. Mostre um caso em que o método de Newton diverge ou oscila, escolhendo deliberadamente um x_0 inadequado para $f(x) = x^3 - 2x + 2$. Explique geometricamente o que acontece.
5. (Aplicação em CC) Implemente uma versão simplificada do algoritmo *Fast Inverse Square Root*: use uma iteração de Newton para refinar uma estimativa inicial de $1/\sqrt{x}$, partindo de uma aproximação grosseira, e compare a velocidade de convergência com o cálculo direto via `math.sqrt`.
6. (Desafio) Generalize o método de Newton para sistemas de duas equações não lineares $f_1(x, y) = 0$, $f_2(x, y) = 0$, usando a matriz Jacobiana. Aplique ao sistema $x^2 + y^2 = 4$, $xy = 1$.

Referências

- BURDEN, R. L.; FAIRES, J. D. *Análise Numérica*. Cengage Learning.
- RUGGIERO, M. A. G.; LOPES, V. L. R. *Cálculo Numérico: Aspectos Teóricos e Computacionais*. Pearson.
- CHAPRA, S. C.; CANALE, R. P. *Métodos Numéricos para Engenharia*. McGraw Hill.
- PRESS, W. H. et al. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press.